

Package ‘SATS’

June 9, 2026

Title Signature Analyzer for Targeted Sequencing (SATS)

Version 1.0.10

Date 2026-06-09

Description Performs mutational signature analysis for targeted sequenced tumors. Unlike the canonical analysis of mutational signatures, SATS factorizes the mutation counts matrix into a panel context matrix (measuring the size of the targeted sequenced genome for each tumor in the unit of million base pairs (Mb)), a signature profile matrix, and a signature activity matrix. SATS also calculates the expected number of mutations attributed by a signature, namely signature burden, for each targeted sequenced tumor. For more details see Lee et al. (2024) <[doi:10.1101/2023.05.18.23290188](https://doi.org/10.1101/2023.05.18.23290188)>.

Imports stats, glmnet, GenomicRanges, IRanges, Biostrings,
dplyr, BSgenome.Hsapiens.UCSC.hg19, BSgenome.Hsapiens.UCSC.hg38

Depends R (>= 4.1.0)

Suggests testthat

License GPL-2

NeedsCompilation yes

Author DongHyuk Lee [aut],
Bin Zhu [aut],
Bill Wheeler [cre]

Maintainer Bill Wheeler <wheelerb@imsweb.com>

Contents

SATS-package	2
CalculateSignatureBurdens	2
EstimateSigActivity	3
GenerateLMatrix	5
GeneratePanelSize	6
GenerateVMatrix	7
MappingSignature	9
ReadBEDAsPanelInfo	10
ReadVCFAsMutationRecord	11
RefTMB	12
SimData	12

Index	14
--------------	-----------

SATS-package

SATS (*Signature Analyzer for Targeted Sequencing*)

Description

This package is created to perform mutational signature analysis for targeted sequenced tumors. Unlike the canonical analysis of mutational signatures, SATS factorizes the mutation counts matrix into a panel context matrix (measuring the size of the targeted sequenced genome for each tumor in the unit of million base pairs (Mb)), a signature profile matrix, and a signature activity matrix. SATS also calculates the expected number of mutations attributed by a signature, namely signature burden, for each targeted sequenced tumor.

Details

This package includes a novel algorithm, SATS, to perform mutational signature analysis for targeted sequenced tumors. The algorithm first applies the `signeR` algorithm to extract profiles of de novo mutational signatures by appropriately adjusting for various panel sizes. Next, the profiles of identified de novo mutational signatures are mapped to the profiles of catalog signatures of tumor mutation burden (TMB), in the unit of the number of mutations per million base pairs, using penalized non-negative least squares. Then, given the panel sizes and profiles of mapped TMB catalog signatures, signature activities are estimated for all samples simultaneously through the Expectation-Maximization (EM) algorithm. Finally, the expected number of mutations attributed by a signature, namely signature burden, is calculated for each targeted sequenced tumor.

The main functions in this package are [GenerateVMatrix](#), [GenerateLMatrix](#), [EstimateSigActivity](#), [CalculateSignatureBurdens](#), and [MappingSignature](#).

The accompanying manuscript applies SATS to 111,711 targeted-sequenced tumors from AACR Project GENIE to construct a real-world, panel-calibrated pan-cancer catalogue of mutational signatures.

Author(s)

Donghyuk Lee <dhyuklee@pusan.ac.kr> and Bin Zhu <bin.zhu@nih.gov>

References

Lee, D., Hua, M., Wang, D., Song, L., Yu, K., Yang, X., Shi, J., Landi, M., Zhu, B. A real-world pan-cancer catalogue of mutational signatures from 111,711 tumors. Submitted.

CalculateSignatureBurdens

Calculate signature burdens

Description

Estimation of the expected number of mutations attributed by TMB-based catalog signatures (signature burden) given the panel size matrix, the catalog signature profile matrix and the signature activities matrix.

Usage

```
CalculateSignatureBurdens(L, W, H)
```

Arguments

L	Panel size matrix or data frame with samples in columns, see GenerateLMatrix
W	Catalog signature profiles matrix or data frame with signatures in columns
H	Activity matrix or data frame with samples in columns, see EstimateSigActivity

Details

The panel size matrix L is of size P (the mutation context) by N (the sample size). The catalog signature profile matrix has dimension of P by K (the number of signatures) and the activity matrix H is of size K by N. For single base substitutions (SBS), P is 96. If K is the number of signatures and N is the number of samples, then H must be of dimension K X N, $\text{ncol}(L) = N$, and $\text{ncol}(W) = K$. When row names and column names are present, `CalculateSignatureBurdens()` checks that L, W and H contain matched mutation contexts, signature names and sample IDs. If the same names are present in a different order, L and W are reordered to match H; if named entries differ between matrices, the function stops with an error. For the catalog signature profile matrix W, reference SBS TMB signature profiles in `data(SimData)` can be used.

Value

A matrix of dimension K X N, where K is the number of signatures and N is the number of samples.

Author(s)

Donghyuk Lee <dhyuklee@pusan.ac.kr> and Bin Zhu <bin.zhu@nih.gov>

See Also

[EstimateSigActivity](#)

Examples

```
data(SimData, package="SATS")

CalculateSignatureBurdens(SimData$L, SimData$TrueW_TMB, SimData$TrueH)
# For more detailed usage, please refer to README and the user manual
# in https://github.com/binzhulab/SATS/tree/main.
```

EstimateSigActivity	<i>Estimate signature activity</i>
---------------------	------------------------------------

Description

Estimation of signature activities given the original mutation type matrix, the panel size matrix, and the catalog signature profile matrix.

Usage

```
EstimateSigActivity(V, L, W, n.start=50, iter.max=5000, eps=1e-5)
```

Arguments

<code>V</code>	Mutation type matrix or data frame with samples in columns
<code>L</code>	Panel size matrix or data frame with samples in columns, see GenerateLMatrix
<code>W</code>	Catalog signature profiles matrix or data frame with signatures in columns
<code>n.start</code>	Number of initializations. The default is 50.
<code>iter.max</code>	Maximum number iterations in the EM algorithm. The default is 5000.
<code>eps</code>	Stopping tolerance in the EM algorithm. The default is 1e-5.

Details

The panel size matrix `L` and mutation type matrix `V` are of size `P` (the mutation context) by `N` (the sample size). The catalog signature profile matrix has dimension of `P` by `K` (the number of signatures). For single base substitutions (SBS), `P` is 96. For the objects `V`, `L`, and `W`, we must have $\dim(V) = \dim(L)$ and $\text{ncol}(W) = K$, where `K` is the number of signatures. When row names and column names are present, `EstimateSigActivity()` checks that `V`, `L` and `W` contain matched mutation contexts and matched sample IDs. If the same names are present in a different order, `L` and `W` are reordered to match `V`; if named entries differ between matrices, the function stops with an error. `EstimateSigActivity()` uses the EM algorithm to estimate signature `n.start`, `iter.max` and `eps` control EM part. Because the convergence to a local saddle point can be an issue of the EM algorithm, it would be good practice to try multiple initial values (`n.start`, the default is 50). For each initial value, the default value of the maximal iteration of the EM algorithm (`iter.max`) is 5000, and the stopping tolerance (`eps`) is set to 1e-5. For the catalog signature profile matrix `W`, reference SBS TMB signature profiles in `data(SimData)` can be used.

Value

A list containing the estimated activity matrix `H`, the log-likelihood `loglike`, and the logical value `converged`.

Author(s)

Donghyuk Lee <dhyuklee@pusan.ac.kr> and Bin Zhu <bin.zhu@nih.gov>

See Also

[CalculateSignatureBurdens](#)

Examples

```
data(SimData, package="SATS")

EstimateSigActivity(SimData$V, SimData$L, SimData$TrueW_TMB)

# For more detailed usage, please refer to README and the user manual
# in https://github.com/binzhulab/SATS/tree/main.
```

GenerateLMatrix	<i>Generate an L Matrix</i>
-----------------	-----------------------------

Description

Generation of the L matrix.

Usage

```
GenerateLMatrix(Panel_context, Patient_Info, Class = c("SBS", "DBS"),
  SBS_order = c("COSMIC", "signeR"), ref.genome = "hg19")
```

Arguments

Panel_context	Either a panel-context matrix returned from GeneratePanelSize or a panel-coordinate data frame containing Chromosome, Start_Position, End_Position and SEQ_ASSAY_ID.
Patient_Info	A sample-panel annotation data frame containing SEQ_ASSAY_ID and either PATIENT_ID or SAMPLE_ID.
Class	A character string specifying the mutation class. Used when Panel_context is a panel-coordinate data frame.
SBS_order	Mutation type order, either "COSMIC" or "signeR". Used for SBS analyses when Panel_context is a panel-coordinate data frame.
ref.genome	The reference genome, either "hg19" or "hg38". Used when Panel_context is a panel-coordinate data frame.

Details

The GenerateLMatrix() function links each sample to its specific sequencing assay, identified by SEQ_ASSAY_ID. For the standard preprocessing workflow, provide panel-coordinate information as the first argument and sample-panel annotation as the second argument. The function then calculates panel-level mutation-context opportunity counts internally and expands them to a sample-level L matrix.

For backward compatibility, the first argument can also be a panel-context matrix returned by [GeneratePanelSize](#). In that lower-level workflow, GenerateLMatrix() only expands the provided panel-context matrix to the sample level.

BED target-region files can first be converted into SATS-compatible panel-coordinate tables with [ReadBEDAsPanelInfo](#).

Value

A data frame (L matrix) of 96 by N, where N is the number of tumors.

Author(s)

Donghyuk Lee <dhyuklee@pusan.ac.kr> and Bin Zhu <bin.zhu@nih.gov>

See Also

[GeneratePanelSize](#), [GenerateVMatrix](#), [ReadBEDAsPanelInfo](#)

Examples

```
data(SimData, package="SATS")

keep <- match(SimData$PatientInfo$SEQ_ASSAY_ID,
              unique(SimData$PanelEx$SEQ_ASSAY_ID), nomatch=0) > 0
PatientInfo <- SimData$PatientInfo[keep, ]

L_mat <- GenerateLMatrix(SimData$PanelEx, PatientInfo, Class="SBS",
                        SBS_order="COSMIC", ref.genome="hg19")

Panel_context <- GeneratePanelSize(genomic_information=SimData$PanelEx,
                                   Class="SBS", SBS_order="COSMIC")
L_mat2 <- GenerateLMatrix(Panel_context, PatientInfo)

stopifnot(identical(colnames(L_mat), colnames(L_mat2)))
stopifnot(identical(rownames(L_mat), rownames(L_mat2)))
# For more detailed usage, please refer to README and the user manual
# in https://github.com/binzhulab/SATS/tree/main.
```

GeneratePanelSize	<i>Generate Panel Size Matrix</i>
-------------------	-----------------------------------

Description

Generation of the panel size matrix given the panel information.

Usage

```
GeneratePanelSize(genomic_information, Class = c("SBS", "DBS"),
                  SBS_order = c("COSMIC", "signeR"), ref.genome="hg19")
```

Arguments

genomic_information	Data frame of panel information (see details).
Class	A character string specifying the mutation class. Must be either "SBS" (for single base substitutions) or "DBS" (for double base substitutions). If "DBS" is selected, the SBS_order argument is ignored.
SBS_order	Mutation type order either one of "COSMIC" or "signeR" (see details).
ref.genome	The reference genome "hg19" or "hg38".

Details

The first argument 'genomic_information' should contain columns 'Chromosome', 'Start_Position', 'End_Position', 'SEQ_ASSAY_ID'. The column 'Chromosome' contains chromosome number where 'Start_Position' and 'End_Position' columns are start and end positions of the targeted panel. The last column 'SEQ_ASSAY_ID' distinguishes different panels consisting of the result. Please note that the column names of 'genomic_information' identical to 'Chromosome', 'Start_Position', 'End_Position', 'SEQ_ASSAY_ID'. BED target-region files can first be converted into this format with [ReadBEDAsPanelInfo](#). The second argument specifies mutation type order as either one of "COSMIC" or "signeR" where "COSMIC" corresponds to the 96 SBS mutation-type order used by

COSMIC v3.2 and "signeR" corresponds to the order from the signeR package. **Note:** The result of 'GeneratePanelSize()' may NOT be an 'L' matrix. For the standard preprocessing workflow, users can call [GenerateLMatrix](#) directly with panel-coordinate and sample-panel annotation data. GeneratePanelSize() is retained as a lower-level helper for users who want to inspect or reuse panel-level context counts. The resulting augmented matrix can be used as the opportunity matrix for 'signeR()' function, 'L' matrix for 'EstimateSigActivity()' and 'CalculateSignatureBurdens()' functions. Therefore, it is important the mutation type order (row names) should be the same as input matrix (Mutation type matrix 'V'). We highly recommend confirming that both 'V' and 'L' matrices have the same mutation type order. The SBS_order argument controls mutation-type ordering only; the COSMIC reference-signature version used for mapping is controlled separately by [MappingSignature](#).

Value

A data frame of 96 by 'S' (the number of panels, 'SEQ_ASSAY_ID') where entries denote the number of trinucleotides per million base pairs.

Author(s)

Donghyuk Lee <dhyuklee@pusan.ac.kr> and Bin Zhu <bin.zhu@nih.gov>

See Also

[GenerateLMatrix](#), [GenerateVMatrix](#), [ReadBEDAsPanelInfo](#), [EstimateSigActivity](#)

Examples

```
data(SimData, package="SATS")
keep <- match(SimData$PatientInfo$SEQ_ASSAY_ID,
              unique(SimData$PanelEx$SEQ_ASSAY_ID), nomatch=0) > 0
PatientInfo <- SimData$PatientInfo[keep, ]

## For the single base substitution (SBS) case:
Panel_context1 <- GeneratePanelSize(genomic_information = SimData$PanelEx, Class = "SBS",
                                   SBS_order = "COSMIC", ref.genome="hg19")
L_mat1 <- GenerateLMatrix(Panel_context1, PatientInfo)
Panel_context2 <- GeneratePanelSize(genomic_information = SimData$PanelEx, Class = "SBS",
                                   SBS_order = "signeR", ref.genome="hg19")
L_mat2 <- GenerateLMatrix(Panel_context2, PatientInfo)

## For the double base substitution (DBS) case:
Panel_context_DBS <- GeneratePanelSize(genomic_information = SimData$PanelEx, Class = "DBS",
                                       ref.genome = "hg19")
L_mat_DBS <- GenerateLMatrix(Panel_context_DBS, PatientInfo)

# For more detailed usage, please refer to README and the user manual
# in https://github.com/binzhulab/SATS/tree/main
```

GenerateVMatrix

Generate a Mutation Count Matrix from Mutation Records

Description

Generate an SBS96 or DBS78 mutation count matrix from a MAF-like mutation record table.

Usage

```
GenerateVMatrix(mutation_record, Class = c("SBS", "DBS"),
               ref.genome = "hg19", mutation_order = NULL)
```

Arguments

<code>mutation_record</code>	A MAF-like data frame containing mutation records.
<code>Class</code>	A character string specifying the mutation class. Must be either "SBS" for single base substitutions or "DBS" for double base substitutions.
<code>ref.genome</code>	The reference genome, either "hg19" or "hg38".
<code>mutation_order</code>	Optional character vector specifying the row order of the returned mutation count matrix. If NULL, SATS uses the row order of <code>RefTMB\$TMB_SBS_v3.4</code> for SBS or <code>RefTMB\$TMB_DBS_v3.4</code> for DBS.

Details

The `mutation_record` data frame must contain the columns `Chromosome`, `Start_Position`, `End_Position`, `Variant_Type`, `Reference_Allele`, `Tumor_Seq_Allele2` and `Tumor_Sample_Barcode`. SBS analyses use records with `Variant_Type == "SNP"` and generate COSMIC-style 96-channel mutation contexts. DBS analyses use records with `Variant_Type == "DNP"` and generate 78 double-base substitution contexts.

This function is intended for MAF-like tabular mutation records. Simple single-sample VCF files can first be converted into this tabular format with [ReadVCFAsMutationRecord](#).

Value

A numeric matrix with mutation contexts in rows and tumor samples in columns.

Author(s)

Donghyuk Lee <dhyuklee@pusan.ac.kr> and Bin Zhu <bin.zhu@nih.gov>

See Also

[GenerateLMatrix](#), [GeneratePanelSize](#), [ReadVCFAsMutationRecord](#)

Examples

```
dir <- system.file("extdata", "refitting_examples", package = "SATS")
sbs_file <- file.path(dir, "SBS_MAF_two_samples.txt")
sbs_mut <- read.table(sbs_file, header = TRUE, sep = "\t", quote = "",
                    stringsAsFactors = FALSE)

V_sbs <- GenerateVMatrix(sbs_mut, Class = "SBS", ref.genome = "hg19")
dim(V_sbs)
```

MappingSignature	<i>Find a subset of TMB-based catalog SBS signatures</i>
------------------	--

Description

This function finds a subset of TMB-based catalog SBS signatures whose linear combination approximate de novo SBS signatures detected by `signeR`.

Usage

```
MappingSignature(W_hat, W_ref=NULL, niter=100, cutoff.I2=0.1, min.repeats=80,
  COSMICv="v3.4")
```

Arguments

<code>W_hat</code>	Matrix or data frame of de novo signatures from <code>signeR</code>
<code>W_ref</code>	NULL or a matrix or data frame of TMB-based catalog signatures. If NULL, then it will default to <code>RefTMB\$TMB_SBS_v3.2</code> or <code>RefTMB\$TMB_SBS_v3.4</code> depending on the value of <code>COSMICv</code> below (see RefTMB).
<code>niter</code>	Number of iterations. The default is 100.
<code>cutoff.I2</code>	Cutoff value to select signatures. The default is 0.1.
<code>min.repeats</code>	Minimum number of iterations to select signatures with $I^2 > \text{cutoff.I2}$. The default is 80.
<code>COSMICv</code>	Version of the TMB-based COSMIC signatures ("v3.2" or "v3.4"). This option is ignored if <code>W_ref</code> is not NULL. The default is "v3.4".

Details

`MappingSignature()` applies penalized non-negative least squares (pNNLS) for selecting the TMB-based catalog signatures. Specifically, it repeats pNNLS 100 times (`niter`) to reduce the randomness of cross-validation involved in pNNLS. Then TMB-based catalog signatures are selected with a coefficient greater than 0.1 (`cutoff.I2`) in more than 80 repeats (`min.repeats`).

Value

A data frame with column names of `W_ref` (it returns COSMIC SBS names if COSMIC catalog based reference signatures are used) and `freq` (the number of repetitions greater than cutoff coefficient values out of `niter` iterations).

Author(s)

Donghyuk Lee <dhyuklee@pusan.ac.kr> and Bin Zhu <bin.zhu@nih.gov>

ReadBEDAsPanelInfo	<i>Convert BED Target Regions to SATS Panel Information</i>
--------------------	---

Description

Convert a Browser Extensible Data (BED) target-region file into the SATS-compatible panel-coordinate table used by [GeneratePanelSize](#) and [GenerateLMatrix](#).

Usage

```
ReadBEDAsPanelInfo(bed_file, seq_assay_id = NULL, has_header = FALSE,
                   chromosome_col = 1, start_col = 2, end_col = 3,
                   seq_assay_col = NULL, name_col = NULL)
```

Arguments

<i>bed_file</i>	Path to a BED file. Files ending in .gz are read with gzfile.
<i>seq_assay_id</i>	Sequencing assay identifier to assign to every interval when <i>seq_assay_col</i> is not supplied.
<i>has_header</i>	Logical value indicating whether the BED file has a header row.
<i>chromosome_col</i>	Column index or name for chromosome.
<i>start_col</i>	Column index or name for the BED start coordinate.
<i>end_col</i>	Column index or name for the BED end coordinate.
<i>seq_assay_col</i>	Optional column index or name containing SEQ_ASSAY_ID.
<i>name_col</i>	Optional column index or name to preserve as Hugo_Symbol.

Details

BED coordinates are 0-based and half-open. SATS panel-coordinate tables use 1-based inclusive coordinates. Therefore this function returns *Start_Position* = BED_start + 1 and *End_Position* = BED_end.

The returned data frame contains at least *Chromosome*, *Start_Position*, *End_Position* and *SEQ_ASSAY_ID*; these columns can be passed directly to [GeneratePanelSize](#) or [GenerateLMatrix](#).

Value

A data frame of panel target intervals in SATS coordinate format.

Author(s)

Donghyuk Lee <dhyuklee@pusan.ac.kr> and Bin Zhu <bin.zhu@nih.gov>

See Also

[GeneratePanelSize](#), [GenerateLMatrix](#), [ReadVCFAsMutationRecord](#)

Examples

```

dir <- system.file("extdata", "refitting_examples", package = "SATS")
bed_file <- file.path(dir, "SATS_example_panel.bed")

panel_info <- ReadBEDAsPanelInfo(
  bed_file = bed_file,
  seq_assay_id = "SATS_EXAMPLE_PANEL",
  name_col = 4
)
head(panel_info)

```

ReadVCFAsMutationRecord

Convert a Simple VCF File to a SATS Mutation Record Table

Description

Convert a simple single-sample Variant Call Format (VCF) file into the MAF-like mutation record table used by [GenerateVMatrix](#).

Usage

```

ReadVCFAsMutationRecord(vcf_file, sample_id = NULL,
  keep_filter = c("PASS", "."))

```

Arguments

vcf_file	Path to a VCF file. Files ending in .gz are read with gzfile.
sample_id	Optional sample identifier for the returned Tumor_Sample_Barcode column. If NULL, SATS uses the single VCF sample column when exactly one is present, or derives a sample identifier from the file name when the VCF has no sample column. Multi-sample VCF genotype parsing is not supported by this lightweight converter.
keep_filter	Character vector of FILTER values to retain. Use NULL to retain all records.

Details

The returned data frame contains the columns required by [GenerateVMatrix](#): Chromosome, Start_Position, End_Position, Variant_Type, Reference_Allele, Tumor_Seq_Allele2 and Tumor_Sample_Barcode.

This function is a lightweight input-preparation utility for standard single-sample targeted-panel VCF files. It does not replace a full clinical variant-normalization pipeline. Multiallelic records are split by ALT allele, but complex normalization, multi-sample genotype parsing, tumor-normal interpretation, phasing and decomposition of complex events should be performed upstream when needed. [GenerateVMatrix](#) uses Variant_Type == "SNP" for SBS analyses and Variant_Type == "DNP" for DBS analyses.

Value

A MAF-like data frame suitable for [GenerateVMatrix](#).

Author(s)

Donghyuk Lee <dhyuklee@pusan.ac.kr> and Bin Zhu <bin.zhu@nih.gov>

See Also

[GenerateVMatrix](#), [ReadBEDAsPanelInfo](#)

Examples

```
dir <- system.file("extdata", "refitting_examples", package = "SATS")
vcf_file <- file.path(dir, "SBS_two_variants.vcf")

mutation_record <- ReadVCFAsMutationRecord(vcf_file)
V <- GenerateVMatrix(mutation_record, Class = "SBS", ref.genome = "hg19")
dim(V)
```

RefTMB

Example Data

Description

The pan-cancer repertoire of reference signatures and the reference TMB (Tumor mutation burden) signature profiles of SBS (Single base substitutions) and DBS (Double base substitutions).

Details

This file consists of the list RefTMB with the following objects:

- TMB_SBS_v3.2 : The version 3.2 reference SBS TMB signature profiles of size 96 by 76.
- TMB_DBS_v3.2 : The version 3.2 reference DBS TMB signature profiles of size 78 by 11.
- TMB_SBS_v3.4 : The version 3.4 reference SBS TMB signature profiles of size 96 by 85.
- TMB_DBS_v3.4 : The version 3.4 reference DBS TMB signature profiles of size 78 by 20.
- SBS_refSigs : The list of cancer specific SBS signature names.
- DBS_refSigs : The list of cancer specific DBS signature names.

SimData

Data for examples

Description

Simulated data as an example

Details

This file consists of the list SimData with the following objects:

- V : Simulated mutation catalog matrix of size 96 by 10027 generated from the Poisson distribution with the mean corresponding to each element of $L \cdot (WH)$, where $\cdot$ is elementwise multiplication, L is the panel size matrix below, W is the profile matrix of the tumor mutation burden (TrueW_TMB below), and H is the signature activity matrix (TrueH below).
- L : Panel size matrix of size 96 by 10027.
- TrueH : Simulated signature activity matrix of size 6 by 10027 used to generate V.
- TrueW_TMB : Tumor mutation burden based signatures size 96 by 6 for 6 SBS signatures (SBS1, SBS2/13, SBS4, SBS5, SBS40, SBS89) used to generate V.
- SingleTumorEx : A single simulated tumor (column singleV) and its sequencing context (column singleL).
- PanelEx: An example sequencing panel information with Chromosome, Start_Position, End_Position, SEQ_ASSAY_ID.
- PatientInfo: An example patient data containing patients ID associated with SEQ_ASSAY_ID.

Index

- * **data**

- RefTMB, [12](#)

- SimData, [12](#)

- * **mutational signatures**

- CalculateSignatureBurdens, [2](#)

- EstimateSigActivity, [3](#)

- * **package**

- SATS-package, [2](#)

CalculateSignatureBurdens, [2](#), [2](#), [4](#)

EstimateSigActivity, [2](#), [3](#), [3](#), [7](#)

GenerateLMatrix, [2–4](#), [5](#), [7](#), [8](#), [10](#)

GeneratePanelSize, [5](#), [6](#), [8](#), [10](#)

GenerateVMatrix, [2](#), [5](#), [7](#), [7](#), [11](#), [12](#)

MappingSignature, [2](#), [7](#), [9](#)

ReadBEDAsPanelInfo, [5–7](#), [10](#), [12](#)

ReadVCFAsMutationRecord, [8](#), [10](#), [11](#)

RefTMB, [9](#), [12](#)

SATS (SATS-package), [2](#)

SATS-package, [2](#)

SimData, [12](#)